Ai programs used:

Claude vs. Gemini

(Note: Claude is set for student use purpose where Gemini isn't)

How I accessed them:

Gemini…gemini.google.com on the browser

Claude…claude.ai on windows

Prompt used:

Files Attached:

solution1.txt

puzzle1.txt

Assignment 4 Instructions.pdf

Complete Step 1 (code generation) for this assignment.

Generated Code:

Claude

```
//
================================================================================
```

```
// Program:      EECS 348 Assignment 4 - Sudoku Solver (Claude version, Step 1)
// Description:  Object-oriented, recursive brute-force (depth-first search with
//               backtracking) Sudoku solver. Finds ALL solutions to a puzzle.
// Inputs:       puzzle1.txt ... puzzle5.txt (9 rows of 9 tokens; '_' = blank)
// Output:       For each file: the file name, the puzzle, and every solution
//               found (or "No solution found").
// Collaborators: None
// Other sources: Generated by Claude (Anthropic) from the assignment text
// Author:       Claude (generated code; to be adapted by the student)
// Creation date: October 16, 2026
//
================================================================================

#include <fstream>   // std::ifstream for reading puzzle files
```

```cpp
#include <iostream>  // std::cout for output
#include <string>    // std::string for file names
#include <vector>    // std::vector to hold solutions

//
------------------------------------------------------------------------
--
// Class: Board
// Holds a 9x9 grid and knows how to check, read, and print itself.
//
------------------------------------------------------------------------
--
class Board {
public:
    static const int N = 9;      // grid dimension
    static const int BLOCK = 3;  // block dimension

    // Start with every cell empty (0 means blank)
    Board() {
        for (int r = 0; r < N; ++r)
            for (int c = 0; c < N; ++c)
                cells[r][c] = 0;
    }

    // Read a puzzle from a file; returns false if the file cannot be read
    bool load(const std::string& fileName) {
        std::ifstream in(fileName.c_str());  // open the file
        if (!in) return false;               // report failure to caller
        for (int r = 0; r < N; ++r) {
            for (int c = 0; c < N; ++c) {
                std::string token;           // each cell is one token
                if (!(in >> token)) return false;  // file too short
                // '_' is a blank (stored as 0); a digit is stored as its
value
                cells[r][c] = (token[0] == '_') ? 0 : (token[0] - '0');
            }
        }
        return true;
    }
```

```
    // Read-only access to a cell
    int get(int r, int c) const { return cells[r][c]; }

    // Write a value into a cell (0 clears it)
    void set(int r, int c, int value) { cells[r][c] = value; }

    // Can 'value' legally go in (r, c)? Checks row, column and 3x3 block.
    bool isValid(int r, int c, int value) const {
        for (int i = 0; i < N; ++i) {
            if (cells[r][i] == value) return false;  // same row
            if (cells[i][c] == value) return false;  // same column
        }
        int br = (r / BLOCK) * BLOCK;  // top row of this block
        int bc = (c / BLOCK) * BLOCK;  // left column of this block
        for (int i = 0; i < BLOCK; ++i)
            for (int j = 0; j < BLOCK; ++j)
                if (cells[br + i][bc + j] == value) return false;  // same
block
        return true;
    }

    // Verify the starting clues do not already break a rule
    bool cluesAreConsistent() {
        for (int r = 0; r < N; ++r) {
            for (int c = 0; c < N; ++c) {
                int v = cells[r][c];
                if (v == 0) continue;
                cells[r][c] = 0;                    // temporarily remove
the clue
                bool ok = isValid(r, c, v);         // is it legal without
itself?
                cells[r][c] = v;                    // put it back
                if (!ok) return false;
            }
        }
        return true;
    }

    // Find the first empty cell in row-major order; false if grid is full
    bool findEmpty(int& r, int& c) const {
```

```
        for (r = 0; r < N; ++r)
            for (c = 0; c < N; ++c)
                if (cells[r][c] == 0) return true;
        return false;
    }

    // Print the grid, digits separated by spaces, '_' for blanks
    void print() const {
        for (int r = 0; r < N; ++r) {
            for (int c = 0; c < N; ++c) {
                if (cells[r][c] == 0) std::cout << "_ ";
                else std::cout << cells[r][c] << " ";
            }
            std::cout << "\n";
        }
    }

private:
    int cells[N][N];  // 0 = empty, 1-9 = digit
};

//
---------------------------------------------------------------------------
--
// Class: SudokuSolver
// Recursive depth-first search with backtracking. Collects every
solution.
//
---------------------------------------------------------------------------
--
class SudokuSolver {
public:
    // Solve a copy of the puzzle and remember all solutions found
    void solve(const Board& puzzle) {
        solutions.clear();               // forget earlier results
        Board working = puzzle;          // work on a copy
        if (!working.cluesAreConsistent()) return;  // contradictory clues
        search(working);                 // start the recursion
    }
```

```
    // Access the solutions found by the last call to solve()
    const std::vector<Board>& getSolutions() const { return solutions; }

private:
    std::vector<Board> solutions;  // every completed grid discovered

    // Recursive step: fill the next empty cell with each legal candidate
    void search(Board& board) {
        int r, c;
        if (!board.findEmpty(r, c)) {    // no empty cell: grid is
complete
            solutions.push_back(board);  // save this solution
            return;
        }
        for (int value = 1; value <= 9; ++value) {  // try each candidate
            if (board.isValid(r, c, value)) {
                board.set(r, c, value);  // place candidate
                search(board);           // recurse with k+1 cells filled
                board.set(r, c, 0);      // backtrack: undo and try the
next
            }
        }
    }
};

//
----------------------------------------------------------------------------
--
// Class: PuzzleRunner
// Loads each puzzle file, runs the solver, and reports the results.
//
----------------------------------------------------------------------------
--
class PuzzleRunner {
public:
    // Process one puzzle file and print everything required
    void run(const std::string& fileName) {
        std::cout << "File: " << fileName << "\n";
        Board puzzle;
        if (!puzzle.load(fileName)) {
```

```
            std::cout << "Could not read puzzle file\n\n";
            return;
        }
        std::cout << "Puzzle:\n";
        puzzle.print();

        SudokuSolver solver;
        solver.solve(puzzle);
        const std::vector<Board>& found = solver.getSolutions();

        if (found.empty()) {
            std::cout << "No solution found\n";
        } else {
            for (size_t i = 0; i < found.size(); ++i) {
                std::cout << "Solution " << (i + 1) << ":\n";
                found[i].print();
            }
            std::cout << "Solutions found: " << found.size() << "\n";
        }
        std::cout << "\n";
    }

    // Process puzzle1.txt through puzzle5.txt
    void runAll() {
        for (int i = 1; i <= 5; ++i)
            run("puzzle" + std::to_string(i) + ".txt");
    }
};

// Entry point: create a runner object and let it do the work
int main() {
    PuzzleRunner runner;
    runner.runAll();
    return 0;
}
```

Gemini

```
/**
 * Program Name: EECS 348 Assignment 4 - Sudoku Solver (Gemini Version)
```

```
 * Description: An object-oriented Sudoku solver using depth-first search
with backtracking.
 * Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
 * Output: Prints the filename, initial puzzle, and solved grid.
 * Author: Student
 * Creation Date: October 15, 2026
 */

#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class Sudoku {
private:
    std::vector<std::vector<char>> board;

    // Helper method to check if placing a digit is valid according to
Sudoku rules
    bool isValid(int row, int col, char digit) const {
        for (int i = 0; i < 9; ++i) {
            if (board[row][i] == digit) return false;
            if (board[i][col] == digit) return false;
        }

        int startRow = row - (row % 3);
        int startCol = col - (col % 3);
        for (int r = 0; r < 3; ++r) {
            for (int c = 0; c < 3; ++c) {
                if (board[startRow + r][startCol + c] == digit) return
false;
            }
        }
        return true;
    }

    // Recursive backtracking solver method
    bool solveSudoku(int row, int col) {
        if (row == 9) return true;
        if (col == 9) return solveSudoku(row + 1, 0);
```

```
        if (board[row][col] != '_') {
            return solveSudoku(row, col + 1);
        }

        for (char digit = '1'; digit <= '9'; ++digit) {
            if (isValid(row, col, digit)) {
                board[row][col] = digit;
                if (solveSudoku(row, col + 1)) return true;
                board[row][col] = '_';
            }
        }
        return false;
    }

public:
    Sudoku() {
        board = std::vector<std::vector<char>>(9, std::vector<char>(9,
'_'));
    }

    // Method to read the puzzle from a file
    bool loadPuzzle(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) return false;

        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                file >> board[i][j];
            }
        }
        file.close();
        return true;
    }

    // Method to print the current state of the board
    void printBoard() const {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                std::cout << board[i][j] << (j == 8 ? "" : " ");
```

```
            }
            std::cout << "\n";
        }
    }

    // Driver method to run the solver and handle output formatting
    void processPuzzle(const std::string& filename) {
        std::cout << filename << "\n";
        printBoard();

        if (solveSudoku(0, 0)) {
            printBoard();
        } else {
            std::cout << "No solution found\n";
        }
        std::cout << "-----------------------------------\n";
    }
};

int main() {
    std::string puzzleFiles[5] = {"puzzle1.txt", "puzzle2.txt",
"puzzle3.txt", "puzzle4.txt", "puzzle5.txt"};

    for (int i = 0; i < 5; ++i) {
        Sudoku puzzleSolver;
        if (puzzleSolver.loadPuzzle(puzzleFiles[i])) {
            puzzleSolver.processPuzzle(puzzleFiles[i]);
        } else {
            std::cout << "Could not open file: " << puzzleFiles[i] <<
"\n";
        }
    }

    return 0;
}
```

Analysis:

1. Correctness
   Gemini's version stops running when it finds the first solution, where Claude is able to output all the solutions.
2. Execution Time
   Gemini technically executes faster, but only because it stops looking for solutions after finding one.
3. Space Complexity
   Same story, gemini uses less space at the cost of not finding every possible solution
4. Maintainability
   Claude has more comments than Gemini does, but most importantly, separates the sudoku reader object and the sudoku solver object. Makes the code easier to manage and read.

Code base: Claude's version

It solves the problem correctly and is easy to maintain/manage.

Edits:

1. Correctness
   Added a couple newlines to make the output cleaner
2. Execution Time
   No change made
3. Space Complexity
   No change made
4. Maintainability
   Added comments and edited header comments (Author: Claude to Author: Jun Brewer)